

Machine Learning II Project Report

The Analysis of the Coffee Quality Dataset

2025-12-31

Contents

1	Introduction	2
1.1	Descriptive Analysis	2
2	Data Preprocessing and Cleaning	4
3	A Brief Mathematical Overview of the ML Methods Used	5
3.1	Support Vector Machine for Regression	5
3.2	XGBoost	6
4	Application of SVM Method	8
5	Application of XGBoost Method	9
6	Literature	9

1 Introduction

This report explores the Coffee Quality dataset, walking through everything from initial descriptive analysis to the math and application of machine learning. Our goal is to use these models to move beyond simple numbers and extract real, meaningful insights into what defines a high-quality cup of coffee.

1.1 Descriptive Analysis

Table 1: Data summary

Name	Piped data
Number of rows	1311
Number of columns	44
Column type frequency:	
character	24
numeric	20
Group variables	None

Variable type: character

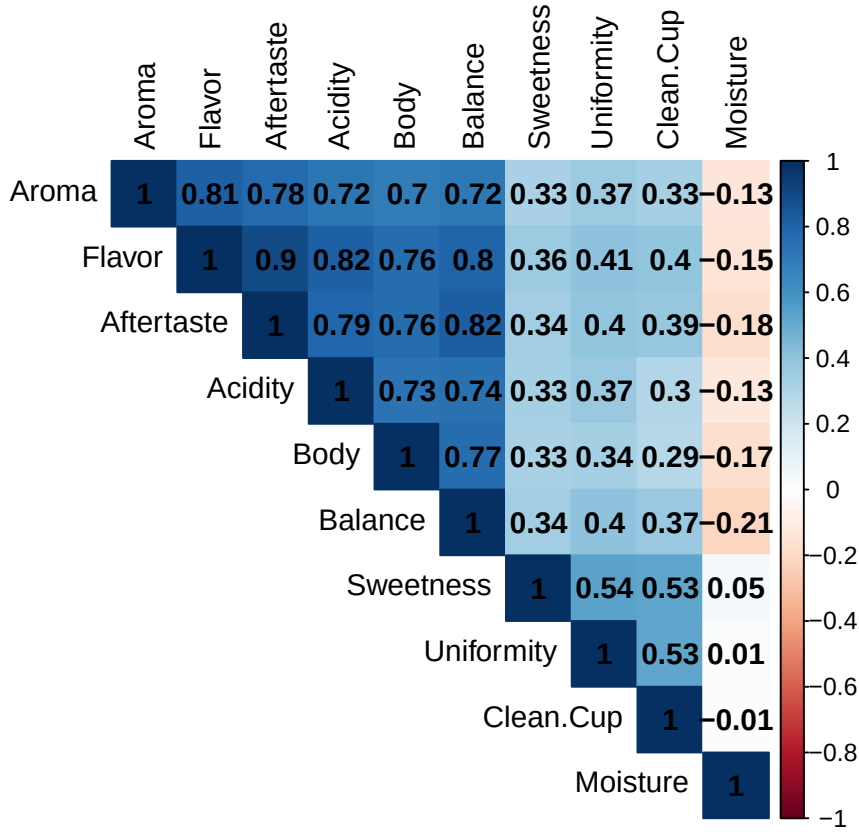
skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Species	0	1.00	7	7	0	1	0
Owner	7	0.99	3	50	0	305	0
Country.of.Origin	1	1.00	4	28	0	36	0
Farm.Name	356	0.73	1	73	0	557	0
Lot.Number	1041	0.21	1	71	0	221	0
Mill	310	0.76	1	77	0	447	0
ICO.Number	146	0.89	1	40	0	841	0
Company	209	0.84	3	78	0	270	0
Altitude	223	0.83	1	41	0	383	0
Region	57	0.96	2	76	0	343	0
Producer	230	0.82	1	100	0	675	0
Bag.Weight	0	1.00	1	8	0	56	0
In.Country.Partner	0	1.00	7	85	0	27	0
Harvest.Year	47	0.96	3	24	0	46	0
Grading.Date	0	1.00	13	20	0	558	0
Owner.1	7	0.99	3	50	0	309	0
Variety	201	0.85	4	21	0	29	0
Processing.Method	152	0.88	5	25	0	5	0
Color	216	0.84	4	12	0	4	0
Expiration	0	1.00	13	20	0	557	0
Certification.Body	0	1.00	7	85	0	26	0
Certification.Address	0	1.00	40	40	0	30	0

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Certification.Contact	0	1.00	40	40	0	27	0
unit_of_measurement	0	1.00	1	2	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
X	0	1.00	656.00	378.60	1	328.50	656.00	983.50	1312.00	
Number.of.Bags	0	1.00	153.89	129.73	0	14.50	175.00	275.00	1062.00	
Aroma	0	1.00	7.56	0.38	0	7.42	7.58	7.75	8.75	
Flavor	0	1.00	7.52	0.40	0	7.33	7.58	7.75	8.83	
Aftertaste	0	1.00	7.40	0.41	0	7.25	7.42	7.58	8.67	
Acidity	0	1.00	7.53	0.38	0	7.33	7.50	7.75	8.75	
Body	0	1.00	7.52	0.36	0	7.33	7.50	7.67	8.58	
Balance	0	1.00	7.52	0.41	0	7.33	7.50	7.75	8.75	
Uniformity	0	1.00	9.83	0.56	0	10.00	10.00	10.00	10.00	
Clean.Cup	0	1.00	9.83	0.77	0	10.00	10.00	10.00	10.00	
Sweetness	0	1.00	9.90	0.53	0	10.00	10.00	10.00	10.00	
Cupper.Points	0	1.00	7.50	0.47	0	7.25	7.50	7.75	10.00	
Total.Cup.Points	0	1.00	82.12	3.52	0	81.17	82.50	83.67	90.58	
Moisture	0	1.00	0.09	0.05	0	0.09	0.11	0.12	0.28	
Category.One.Defects	0	1.00	0.43	1.83	0	0.00	0.00	0.00	31.00	
Quakers	1	1.00	0.18	0.84	0	0.00	0.00	0.00	11.00	
Category.Two.Defects	0	1.00	3.59	5.35	0	0.00	2.00	4.00	55.00	
altitude_low_meters	227	0.83	1759.55	8767.85	1	1100.00	1310.64	1600.00	190164.00	
altitude_high_meters	227	0.83	1808.84	8767.19	1	1100.00	1350.00	1650.00	190164.00	
altitude_mean_meters	227	0.83	1784.20	8767.02	1	1100.00	1310.64	1600.00	190164.00	

Correlation Between Sensory Attributes



Summary of the Descriptive Analysis: This dataset includes 1,311 coffee evaluations defined by 44 variables, covering both sensory scores and environmental data. For our analysis, we chose Cupper Points as the target variable rather than Total Cup Points, as the latter is simply an aggregate sum of the other sensory variables. A major focus of our preprocessing was handling missing values and inconsistencies in key predictors: 17% of altitude data was missing, and the unit of measurement variable was critical for standardizing weights across the dataset.

2 Data Preprocessing and Cleaning

Feature selection: In SVM Regression, categorical variables are One-Hot Encoded using dummy variables. This means, features with many unique values will lead to a sparse feature matrix. This is disadvantageous as it increases computational cost and poses the risk of overfitting. Additionally, categories with only a few or even only one observation most likely will not add to model robustness, but are adding noise to the signal. Therefore, we are removing categorical features with many unique values, only keeping columns with a low number of unique values. E.g. we are not including the “Region” column with 344 unique values in the model training, because we’d on average have less than 4 observations per Region. For features with a high number of missing values, we either remove the column or impute the missing values. For instance for the “Variety” column, we have about 15% missing values and 8% entries “other” at 30 different varieties. Keeping those, would mean adding 30 sparse columns, and at the same time the information gain is most likely not too high with this many missing values. Therefore, we are not including this feature.

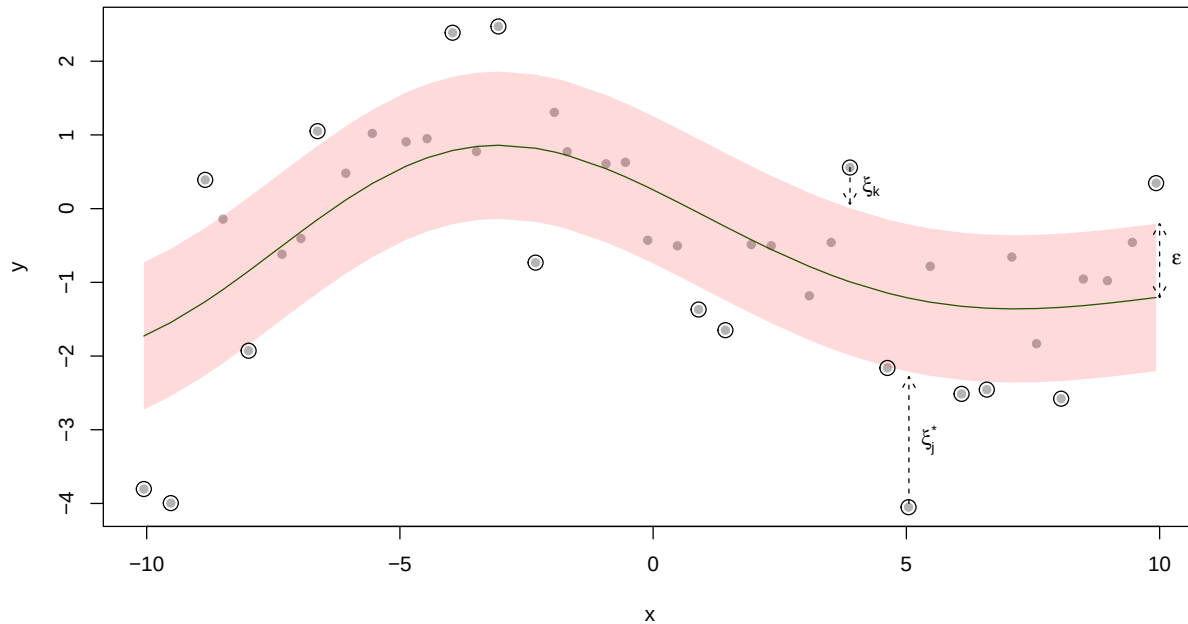
Imputation: For the altitude column, we have about 17% missing values. Here, we are imputing the missing values from the “Region” column. For each observation with missing altitude, we are imputing the median altitude of all coffees from the same region. This covers almost all missing values. For the remaining observations, we are imputing the mean altitude of all coffees from the same country of origin. As the “Region” and “Country” columns are not used in the model training, we are not creating correlated features with this imputation. We are imputing the missing values after the train/test split, using only values from the training set, to avoid data leakage from the test set into the training set.

Further preprocessing: The bag weight column contains both pounds and kilograms, so we are converting all to kilograms. However, for some entries, it is unclear which unit is used. For these, we are averaging the weight in pounds and kilograms. This means for those entries the weight is certainly incorrect, but should be within an order of magnitude of the correct weight. If we can assume that speciality coffee is sold in smaller quantities, we can still gain valuable information from the weight.

Scaling: The SVM method used for model training has a built in scaling functionality. Both predictor and target variables are scaled to have expected value 0 and variance 1. [<https://rdr.io/rforge/e1071/man/svm.html>] This transformation is based on the training data and applied to train and test data sets.

3 A Brief Mathematical Overview of the ML Methods Used

3.1 Support Vector Machine for Regression



Support Vector Regression expands support Vector Machines from discrete classification to a con-

tinuous regression model.

For the regression task, instead of fitting a hyperplane, we are trying to fit a (multidimensional) function $f(x)$ to the data. Similar as in the classification task, we draw a margin around the function. However, now the margin ε is fixed at the start. (Note that ε here, is not the slack variable as in Hasties and Tibshirani, but rather defines the width of the margin. Also, we don't aim to optimize the margin anymore, but instead fix it before the computation starts). We want to find a function $f(x)$ that is as flat as possible, while fitting all points into the required margin. Again, we allow for some slack using slack variables. A wider margin decreases the risk of overfitting, while a smaller margin captures more intricacies of the data.

Mathematically speaking, want to find a function $f(x)$ with $|f(x_i) - y_i| \leq \varepsilon$ for all i . To avoid overfitting, we also introduce the constraint of making the function as flat as possible. "Flatness" in this context is a measure of how sensitive the function is to change. For a linear function $f(x) = x^\top \beta + b$ increasing this flatness can be expressed through minimizing the norm of the function gradient: $\|\beta\|$. Minimizing the expression $\frac{1}{2}\|\beta\|^2$ leads to the same optimum, while allowing for more elegant mathematical solutions, and is therefore used for computation.

Again, we want to allow for some slack, so as in the SVM for classification, we are introducing slack variables ξ_i and ξ_i^* . These slack variables allow for some room for margin violations. The optimization problem we want to solve is to minimize:

$$\frac{1}{2}\|\beta\|^2 + K \sum_{i=1}^n (\xi_i + \xi_i^*)$$

subject to:

$$\begin{aligned} y_i - f(x_i) &\leq \varepsilon + \xi_i, \\ f(x_i) - y_i &\leq \varepsilon + \xi_i^* \end{aligned}$$

and

$$\xi_i > 0, \xi_i^* > 0 \quad \forall i$$

where K is fixed and β , ξ and ξ^* are variable. The factor K is introduced as cost for regularization purposes. It is chosen a priori to determine how strong the tradeoff between flatness and overfitting should be. A larger K leads to a stronger impact of the term, therefore penalizing stronger margin violations more heavily and resulting in a less flat function curve (higher bias, lower variance). Conversely, a smaller K leads to a flatter curve (lower bias, higher variance).

As in the SVM for classification, this can be generalized to non-linear functions by transforming the features using kernels.

3.2 XGBoost

XGBoost is an algorithm based on gradient boosted trees and second-order approximation (Chen 2016). It is widely used in machine learning due to its flexibility and strong empirical performance. These properties arise from a combination of algorithmic and systems-level optimizations, including efficient handling of sparse input data, approximate tree construction via weighted quantile sketches, and parallelized tree learning. In addition, careful optimization of memory access, data compression, and out-of-core computation enables the method to scale to very large datasets while maintaining computational efficiency.

This section describes the mathematical foundations of the algorithm, namely ensemble methods, regularization, and gradient boosting machines combined with second order approximation.

3.2.1 Tree Ensembles and Regularization

XGBoost models the prediction as an additive ensemble of K functions:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F},$$

where each f_k is a regression tree belonging to the function space $\mathcal{F}$ (Chen 2016; James et al. 2021). Each tree acts as a weak learner that partitions the feature space into disjoint regions and assigns a constant prediction to each leaf (James et al. 2021).

The model is trained by minimizing a regularized objective function of the form

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k),$$

where $\ell(\cdot)$ is a differentiable loss function and $\Omega(f)$ is a regularization term that penalizes model complexity (Chen 2016). This term is defined as

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

with T denoting the number of leaves in the tree and w_j the weight of leaf j . The regularization term encourages simpler trees and smooth leaf weights, helping to prevent overfitting.

The ensemble structure allows the model to reduce bias by combining multiple weak learners, while regularization controls variance by limiting tree complexity and the magnitude of leaf weights (James et al. 2021). This balance between bias and variance enables XGBoost to achieve strong predictive performance while maintaining good generalization.

3.2.2 Gradient Boosting Machines

The tree ensemble is constructed in a stage-wise manner, where weak learners are added iteratively to improve the model (Friedman 2001). At iteration t , the prediction is given by

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i),$$

with $f_t \in \mathcal{F}$ denoting the newly added regression tree.

Gradient tree boosting can be interpreted as gradient descent in function space, where each new learner is chosen to minimize the loss function in the direction of the negative gradient (Friedman 2001). Specifically, the new tree at iteration t is trained to fit the negative first order gradient

$$-g_i^{(t)} = - \left. \frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{y}_i^{(t-1)}}.$$

XGBoost extends this framework by explicitly approximating the objective function using a second-order Taylor expansion of the loss around the current predictions:

$$\ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx \ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2,$$

where

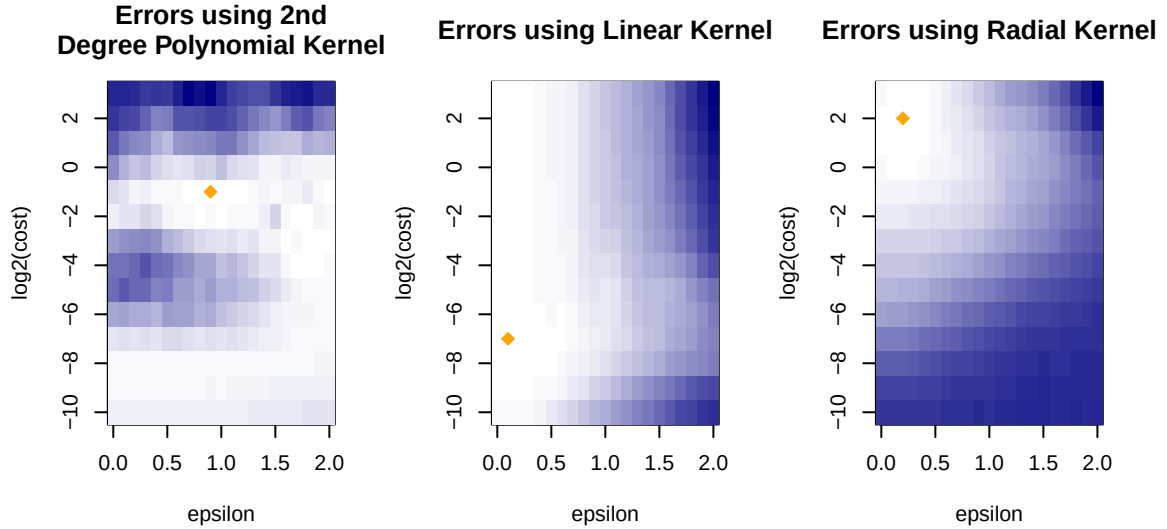
$$h_i = \left. \frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y} = \hat{y}_i^{(t-1)}}$$

is the second order derivative of the loss (Chen 2016). By incorporating both first- and second-order information, XGBoost enables efficient optimization of tree structures and leaf weights within each boosting iteration.

4 Application of SVM Method

4.0.1 Support Vector Regression - Hyperparameter Optimization

For the prediction of copper points we train an SVM for regression on the data set, holding out the test data set for final evaluation. To choose appropriate hyperparameters, we conduct a hyperparameter optimization on the training data set using 10-fold cross-validation. We use a grid search for different values for cost and epsilon for a radial, linear, and 2-degree polynomial kernel. We compare the mean absolute error between runs to choose the best-performing hyperparameter combination. Finally, we train a model with these hyperparameters on the entire training set and evaluate on the test data set.

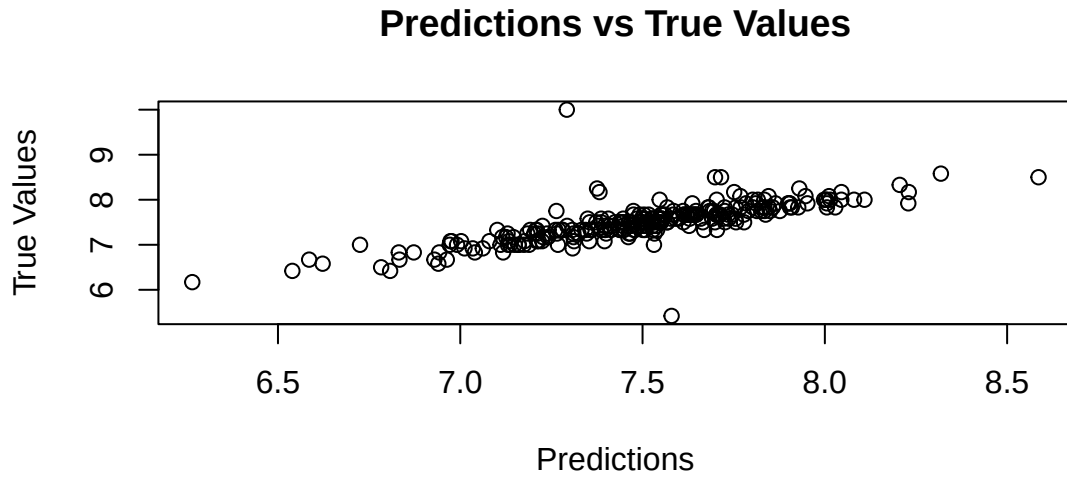


These figures show the mean absolute error for the different hyperparameters by kernel. The best performing hyperparameter combination for each kernel are marked with orange points.

Best SVM kernel and parameters selected:

```
## kernel epsilon cost
## 2 linear      0.1 0.0078125
## [1] "MAE: 0.143099275160558"
```

The best performing kernel is the linear kernel, which means the relationship between the features and the target variable is best approximated by a multidimensional hyperplane.



Here, we can see the predictions vs. the true values for the test data set. We can see that aside from two outliers, the predictions are close to the true values.

5 Application of XGBoost Method

6 Literature

- Chen, Tianqi. 2016. "XGBoost: A Scalable Tree Boosting System." *Cornell University*.
- Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 1189–1232.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed. Springer. <https://doi.org/10.1007/978-1-0716-1418-1>.